Національний технічний університет України «КПІ ім. Ігоря Сікорського»

Лабораторна робота №2
з дисципліни «Розробка мобільних застосувань під Android»

на тему
«ДОСЛІДЖЕННЯ РОБОТИ З КОМПОНЕНТОМ FRAGMENT»

Виконав:
студент групи ЗПІ-ЗП-31
Сікорський О.С.

Викладач:
к.т.н.
Орленко С.П.

Київ – 2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набути практичні навички з використання фрагментів для інтерфейсу користувача.

Завдання: Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («OK»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

Завдання згідно з варіантом №14: Вікно містить згорнутий список книг (автори), групу опцій (роки видання), тобто радіо-батони, та кнопку «OK». Вивести інформацію щодо вибору.

Виконання:

Роботу:
- виконано в середовищі Android Studio з використанням Android SDK 36;
- протестовано на реальному пристрої Lenovo Tab P12 (Android 15).

Роботу виконано мовою програмування Kotlin з дотриманням одноекранної архітектури (Single-Activity Architecture). Усі взаємодії з фрагментами координуються головним контейнером — MainActivity.

Згідно з вимогами, інтерфейс було розділено на два незалежних модульних компоненти (фрагменти):

1. **InputFragment (Перший фрагмент):** Відповідає за збір даних від користувача. Він містить згорнутий список (Spinner) для вибору книги, групу радіо-кнопок (RadioGroup та RadioButton) для вибору року видання та кнопку «OK» для підтвердження вибору.
2. **OutputFragment (Другий фрагмент):** Відповідає за відображення результату. Він містить текстове поле (TextView) з відформатованим результатом (Книга + Рік) та кнопку «Cancel», яка закриває цей фрагмент та очищає дані у першому.

Взаємодія між компонентами реалізована за схемою **Fragment -> Activity (через Interface) -> Fragment (через Bundle)**, що гарантує низьку зв'язність

(loose coupling) компонентів додатка. Це відповідає правилам архітектури, оскільки один фрагмент не повинен безпосередньо маніпулювати іншим.

Ключові методи Відповідно до архітектури застосунку, логіка роботи розподілена між класами. Основні ключові методи:

- **У класі** MainActivity.kt**:**
  - onDataPassed(text: String): реалізація інтерфейсу (callback) для отримання сформованого рядка даних з InputFragment та передачі їх в OutputFragment за допомогою об'єкта Bundle.
  - onCancelClicked(): реалізація інтерфейсу для обробки натискання кнопки «Cancel». Метод видаляє OutputFragment зі стеку переходів назад за допомогою виклику popBackStack() та звертається до InputFragment для очищення форми.
- **У класі** InputFragment.kt**:**
  - onCreateView(...): метод життєвого циклу фрагмента, який викликається для промальовування інтерфейсу і повертає об'єкт View, що є кореневим у макеті фрагмента.
  - setupSpinner(): налаштовує ArrayAdapter для випадаючого списку (Spinner) та прив'язує масив книг.
  - handleOkButtonClick(): здійснює валідацію вибору користувача (перевіряє, чи обрано RadioButton у RadioGroup) та передає відформатований рядок в Activity через інтерфейс OnDataPassedListener.
  - clearForm(): публічний метод, що скидає стан віджетів (Spinner та RadioGroup) до початкового стану.
- **У класі** OutputFragment.kt**:**
  - displayReceivedData(): метод, що витягує передані дані з аргументів (Bundle) та встановлює їх у текстове поле TextView.
  - setupListeners(): встановлює обробник події на кнопку «Cancel», який через інтерфейс сповіщає MainActivity про необхідність закриття поточного фрагмента.

Висновок:

У ході виконання лабораторної роботи було успішно досліджено роботу з компонентом Fragment в операційній системі Android. Було створено застосунок з одноекранною архітектурою, що складається з двох фрагментів: для вводу та виводу даних. Набуто практичних навичок з використання FragmentManager та FragmentTransaction, реалізовано низькозв'язну взаємодію між фрагментами за допомогою інтерфейсів зворотного виклику (callbacks). Також налаштовано коректну роботу зі

стеком переходів назад (Back Stack), що дозволяє плавно керувати екранами та станом віджетів.

Відповіді на контрольні питання:

1. Призначення та можливості компоненту Фрагмент. Призначення: Фрагмент (Fragment) представляє поведінку або модульну частину інтерфейсу користувача всередині Діяльності (Activity)

Його головне призначення — забезпечення більшої динамічності та гнучкості користувацьких інтерфейсів, особливо на пристроях із великими екранами, таких як планшети

Можливості:

Об'єднання та повторне використання: Можна об'єднувати декілька фрагментів в одній Діяльності для побудови багатопанельного інтерфейсу, а також повторно використовувати один і той самий фрагмент у різних Діяльностях.

Незалежність: Фрагмент самостійно обробляє власні події введення і може бути доданий або видалений незалежно від інших елементів, поки Діяльність знаходиться у стані «відновлена».

Фонова робота: Фрагмент не зобов'язаний бути частиною візуального екрану Діяльності; його можна використовувати без інтерфейсу як невидимий робочий потік.

Навігація: Операції з фрагментами можна додавати до стеку переходів назад (back stack), що дозволяє користувачеві скасовувати транзакції з фрагментами за допомогою кнопки "Назад".

2. Життєвий цикл компонента Фрагмент. Фрагмент має власний життєвий цикл, на який безпосередньо впливає життєвий цикл батьківської Діяльності. Він може перебувати у трьох основних станах: Відновлено (resumed - фрагмент видимий і активний), Призупинено (paused - видимий, але фокус має інша Діяльність) та Зупинено (stopped - невидимий, але зберігає свій стан). Основні методи зворотного виклику життєвого циклу включають:

onAttach(): викликається, коли фрагмент зв'язується з Діяльністю.

onCreate(): початкова ініціалізація фрагмента.

onCreateView(): створення та промальовування візуального макета (повертає об'єкт View).

onActivityCreated(): викликається після завершення onCreate() батьківської Діяльності.

onStart() та onResume(): фрагмент стає видимим та повністю активним.

onPause(): викликається, коли користувач залишає фрагмент (тут слід фіксувати/зберігати поточні зміни).

onStop(): фрагмент стає невидимим.
onDestroyView(): видалення ієрархії представлень фрагмента.
onDestroy() та onDetach(): остаточне знищення фрагмента та розрив його зв'язку з Діяльністю.
3. Способи створення компонента Фрагмент. У програмному коді створення фрагмента відбувається шляхом опису підкласу базового класу Fragment. Також можна розширювати спеціалізовані існуючі підкласи: DialogFragment (для переміщуваних діалогових вікон), ListFragment (для відображення списків) або PreferenceFragment (для створення вікон налаштувань). Для інтеграції фрагмента в користувацький інтерфейс Діяльності існують два способи:
Декларативний: оголошення фрагмента безпосередньо у файлі макета (XML) Діяльності за допомогою тегу <fragment>.
Програмний: додавання фрагмента в існуючий об'єкт ViewGroup (контейнер) під час виконання програми за допомогою FragmentManager та FragmentTransaction.
4. Опишіть способи управління компонентом Фрагмент. Управління фрагментами в межах Діяльності здійснюється за допомогою класу FragmentManager. Він дозволяє:
Пошук фрагментів: отримувати посилання на існуючі фрагменти за допомогою методів findFragmentById() або findFragmentByTag().
Виконання транзакцій: проводити набір одночасних операцій (таких як add(), remove(), replace()) за допомогою об'єкта FragmentTransaction. Для застосування цих змін до Діяльності викликається метод commit().
Керування стеком переходів: додавати транзакції до стеку переходів назад викликом addToBackStack() (викликається до commit()), що дозволяє системі скасувати зміни при натисканні кнопки "Назад". Видалити фрагмент зі стеку можна методом popBackStack().
Моніторинг: реєструвати слухачів для відстеження змін у стеці переходів через addOnBackStackChangedListener().
5. Опишіть способи взаємодії між Фрагментами. Для збереження модульності один фрагмент не повинен безпосередньо маніпулювати іншим. Замість цього взаємодія реалізується за класичною схемою: Фрагмент -> Діяльність (через Інтерфейс) -> Фрагмент.
Механізм роботи:
Усередині фрагмента визначається інтерфейс зворотного виклику (callback interface), який батьківська Діяльність зобов'язана реалізувати.
У методі onAttach() фрагмент отримує посилання на цей інтерфейс, виконавши приведення типу об'єкта Activity.

При настанні події фрагмент викликає метод цього інтерфейсу, передаючи дані в Діяльність.
Діяльність, отримавши дані, за необхідності знаходить інший фрагмент (через FragmentManager) і передає йому інформацію (наприклад, через Bundle) або сповіщає про необхідність оновлення екрану.
6. Наведіть поняття системи, малої системи та мобільної платформи. У наданих джерелах та історії розмови відсутні прямі теоретичні визначення цих загальних понять. Матеріали лекцій зосереджені виключно на особливостях системи Android, яка описується як багатофункціональна платформа, що має чотирирівневу архітектуру: ядро Linux, рівень системних/функціональних бібліотек (включно із середовищем Dalvik/ART), каркас додатків та рівень самих додатків.
7. Опишіть типи мобільних застосунків.
1. Нативні застосунки (Native Android)
- Опис: Створюються виключно для операційної системи Android.
- Роль Kotlin: Це офіційна та пріоритетна мова (First-Class) від Google для розробки під Android.
- Особливості: Максимальна швидкодія, плавні анімації, повний і прямий доступ до апаратних можливостей пристрою (Bluetooth, камера, сенсори) та системних API.
- Інструменти UI: Jetpack Compose або класичний XML.

2. Кросплатформні застосунки (Kotlin Multiplatform / KMP)
- Опис: Створюються одночасно для кількох платформ (Android, iOS, Web, Desktop) з використанням спільної кодової бази.
- Роль Kotlin: Мова використовується для написання єдиної бізнес-логіки (робота з мережею, бази даних, алгоритми). Інтерфейс (UI) при цьому можна робити нативним для кожної платформи (наприклад, Swift для iOS) або використовувати спільний фреймворк Compose Multiplatform.
- Особливості: Значна економія часу на розробку та підтримку, оскільки не потрібно писати бізнес-логіку двічі для різних ОС, при цьому зберігається близька до нативної продуктивність.

Інші типи (для загальної класифікації): Існують також гібридні застосунки (по суті вебсайти, загорнуті в контейнер WebView). Хоча їхня Android-оболонка може бути написана на Kotlin, сама логіка та інтерфейс створюються за допомогою вебтехнологій (HTML, CSS, JavaScript), тому Kotlin відіграє там мінімальну роль.

8. Наведіть класифікацію та загальну характеристику середовищ розробки мобільних

застосунків.

Середовища розробки мобільних застосунків класифікують за їхнім цільовим призначенням:

- Нативні IDE: Офіційні інструменти, створені розробниками самих операційних систем. Надають повний набір інструментів (профайлери, емулятори) для конкретної ОС і гарантують найвищу швидкодію застосунків.
    - *Android Studio* (Google) — стандарт для Android (Kotlin, Java).
    - *Xcode* (Apple) — ексклюзив для iOS/macOS (Swift, Objective-C).
- Універсальні / Кросплатформні: Гнучкі редактори коду, які за допомогою плагінів адаптуються для розробки під різні платформи одночасно (використовуючи Flutter, React Native, KMP).
    - *Visual Studio Code*, *IntelliJ IDEA* — легші, мають широку підтримку екосистеми плагінів.
- Low-Code / No-Code та хмарні середовища: Платформи для візуальної розробки (часто у браузері) з мінімальним написанням коду або його генерацією.
    - *FlutterFlow*, *AppGyver* — дозволяють швидке прототипування та випуск простих застосунків.

9. Наведіть класифікацію та загальну характеристику мобільних платформ.

Сучасні мобільні ОС класифікують за типом їхньої екосистеми та ліцензування:

- Відкриті (Open-Source) / Фрагментовані (наприклад, Android):
    - *Характеристика:* Базуються на ядрі Linux (AOSP). ОС є гнучкою, дозволяє глибоку кастомізацію та встановлення застосунків із будь-яких джерел.
    - *Особливість:* Встановлюється на пристрої сотень різних виробників, що створює сильну фрагментацію (величезна кількість комбінацій екранів, процесорів та версій ОС), яка ускладнює тестування.
- Закриті екосистеми (наприклад, iOS від Apple):
    - *Характеристика:* Пропрієтарна система з закритим кодом, яка встановлюється виключно на пристрої одного виробника.
    - *Особливість:* Забезпечує ідеальну оптимізацію програмного забезпечення під апаратну частину, високий рівень безпеки та жорсткий контроль (через App Store). Немає проблеми фрагментації.
- Альтернативні / Мікроядерні (наприклад, HarmonyOS):
    - *Характеристика:* Сучасні ОС, що будуються не як суто телефонні, а як платформи для всього "Інтернету речей" (IoT)

— від смарт-годинників до автомобілів, забезпечуючи безшовну взаємодію між ними.